Normally, when using `argparse`, you are responsible for instantiating a parser, but the `BaseCommand` class does this for you. When working with a Django management command, you override the `.add_arguments()` method, which gets the parser as parameter. Inside your `.add_arguments()` method, you register any command-line positional arguments and flags you want for your management command.

Consider adding three filter flags to our `musicians` management command: one for the birth year and one for each the first and last name. To do this, you need to call the parser object's `.add_argument()` method three times: once for each flag. The first parameter to `.add_argument()` is the name of the argument you're registering. You indicate it is a flag by naming it with leading dashes (as opposed to a positional argument, which has no dashes). For flags, you can optionally provide a second parameter with the short-form name of the flag. To add a flag for the musician's last name, the first two parameters to `.add_argument()` are `--last_name` and `-l`.

The parser's `.add_argument()` method takes 11 different parameters, allowing you to finely control how many instances of an argument there are, default values, what type the string should be converted into, and much more. For our `musicians` example, let's keep it simple and just add the `help` parameter. The string value of `help` gets included in the help screen for the management command, explaining how your flag gets used.

If the only parameters you pass to `.add_argument()` are the name of the flag and the `help` option, your flag gets configured to expect a string value after it, which, in this case, is the data used to filter by last name.

Recall that the `.handle()` method takes `*args` and `**options` in its signature. When you implement `.add_arguments()`, the `options` parameter can be treated like an `argparse` namespace. That's the dictionary-like object that contains the positional arguments and flags passed to your management command.

To implement a filter based on a musician's last name, you first check the contents of `options["last_name"]`. If the `--last_name` (or `-l`) flags got used, the value passed to the flag gets included in `options`. If the flag wasn't present, then `options["last_name"]` contains `None`.

Back in chapter 4, you learned how to write database queries using the `.filter()` method. In particular, you can modify a query using double-underscore field-lookup modifiers. Passing `last_name__gte=value` to the `Musician` query returns all the `Musician` objectss with last names greater than or equal to the provided value. If `options["last_name"]` isn't empty, then you filter based on its content.

> **NOTE** To keep things simple, the name queries are case sensitive; otherwise, you have to deal with the fact that ASCII $a > A$. Querying a last name beginning with *Z* includes any last names that start with a lowercase letter.

Writing a filter based on first names is the same as last names, except using a `--first_name` flag and a `first_name__gte` field-lookup modifier. Filtering by birth date is a little trickier. To filter a database column that contains a date, you need to